Detection of SQL Antipatterns in jOOQ Database Access Code Using Large Language Models

Kristo Isberg
Supervisor: Erki Eessaar, PhD

Tallinn University of Technology
25.05.2026

SQL Antipatterns

- "... a commonly occurring solution to a problem that generates decidedly negative consequences" (Brown et al., 1998)
- "... the most frequent missteps software developers naively make while using SQL" (Karwin, 2022)
- Can affect performance, maintainability, portability, and data integrity

What is jOOQ?

- Java Object Oriented Querying
- Open-source Java library for interacting with SQL databases
- Highly popular, especially in industrial projects
- Integrates SQL into Java as a domain-specific language
- Leverages code generation heavily
- Provides strong type safety

SQL Antipatterns in Java: literature review

- Quantifiable performance impact
  - Fixing *Implicit Columns* improved runtime and energy efficiency of local queries in mobile applications by more than 25%
- Prevalent in Java projects
  - *Implicit Columns* affects every 50th query
  - Remain unfixed longer than traditional code smells
  - Authors suspect lack of awareness or low priority
- Two tools capable of statically detecting SQL Antipatterns in Java
  - None capable of detecting from jOOQ DSL

Research Goals

- Development of an LLM-based static analysis tool for detecting SQL antipatterns
- Studying the prevalence and occurrence patterns of SQL antipatterns in jOOQ

Research Questions

- **RQ1:** How do different LLMs compare in identifying SQL antipatterns in Java code that uses jOOQ for database access?
- **RQ2:** How do different prompting strategies affect the detection performance of LLMs in identifying SQL antipatterns in Java code that uses jOOQ for database access?
- **RQ3:** How accurately can the developed LLM-based tool detect SQL antipatterns in Java code that uses jOOQ for database access?
- **RQ4:** What patterns can be observed in the occurrence of SQL antipatterns in Java code that uses jOOQ for database access?
- **RQ5:** Which jOOQ API methods are most frequently associated with query antipattern occurrences?

Dataset Creation

- Used GitHub API to mine for projects using jOOQ
  - Filtered out non-Java projects, duplicates, etc
  - Gathered **602** applicable projects
- Annotated a subset of 10% projects
  - Stratified sampling using head-tail breaks to preserve size distribution
  - 4 large, 10 medium, 47 small projects
  - **1,562** total antipattern occurrences identified in them

Dataset Creation

- Internal consistency measured using Cohen's Kappa
  - Re-annotated 10% of files after one month

$$\kappa = \frac{p_o - p_e}{1 - p_e} = 0.834$$

  - Result: *Almost perfect agreement*
- **61** projects split into training/validation/test sets
  - Split ratio of 34/33/33
  - Monte Carlo optimisation to achieve an optimal split

Analysis of LLMs and prompting strategies

- Models:
  - OpenAI GPT-5.2 — reasoning model
  - Anthropic Claude 4.5 Opus — non-reasoning* model
  - Z.ai GLM-5 — open model
  - OpenAI gpt-oss-120B — medium-sized open model
- Prompting strategies:
  - Zero-Shot — baseline
  - Few-Shot — providing examples
  - Chain-of-Thought — "Think step by step."
  - Tree-of-Thought — simulating a conversation between three experts

Analysis of LLMs and prompting strategies

- Designed prompts for each strategy, evaluating against training set
- Evaluated performance of each LLM & strategy combination on validation set

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \quad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \quad F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

- Localisation task: True Positive only if model found the correct lines according to Intersection over Union

$$\mathrm{IoU} = \frac{|S_p \cap S_g|}{|S_p \cup S_g|} \geq 0.5$$

RQ1: How do different LLMs compare in identifying SQL antipatterns in Java code that uses jOOQ for database access?

- All large models demonstrated similar performance
  - Varying costs and runtimes
  - GPT-5.2 was more paranoid than others
- gpt-oss-120B suffered from anomalies degrading performance

Model	Precision	Recall	F1-Score	Cost (USD)	Runtime (s)
GPT-5.2 (reasoning)	0.85	**0.92**	**0.88**	26.16	2,423
GLM-5 (reasoning)	**0.88**	0.89	**0.88**	11.11	5,775
Claude Opus 4.5 (non-reasoning)	**0.88**	0.89	**0.88**	29.97	**367**
gpt-oss-120B (reasoning)	0.83	0.87	0.83	**1.49**	2,808

RQ2: How do different prompting strategies affect the detection performance of LLMs in identifying SQL antipatterns in Java code that uses jOOQ for database access?

- 2% performance gains to non-reasoning model from CoT and ToT
- Up to 1% gains to open models from Few-Shot
- Gains for one model generally balanced by degradations for other models
- Large increases to cost and runtime

Strategy	Avg F1-Score Increase	Avg Cost Increase	Avg Runtime Increase
Few-Shot	0.6%	31%	-1%
Chain-of-Thought	-0.3%	23%	90%
Tree-of-Thought	0.1%	88%	638%

Analysis tool development and evaluation

- **Stack:** Bun (build tool & runtime), TypeScript, Vercel AI SDK.
- **Architecture:** Self-contained CLI tool
- **Pre-processing:**
  - Irrelevant files skipped based on black- and whitelists
  - **Line number prefixing:** Helps LLMs maintain spatial awareness and prevent miscalculations
- Zero-Shot prompt, default model Claude Opus 4.5 (non-reasoning)
- **Output:** Human-readable text summaries + machine-readable JSON/CSV.

RQ3: How accurately can the developed LLM-based tool detect SQL antipatterns in Java code that uses jOOQ for database access?

$$P = 0.88, \quad R = 0.88, \quad F_1 = 0.88$$

- **Best performers:** *Implicit Columns* $(F_1 = 0.96)$ and *31 Flavors* $(F_1 = 0.97)$
  - **Why?** Self-contained; specific detection patterns
- **Worst performers:** *Keyless Entry* $(F_1 = 0.69)$, *Poor Man's Search Engine* $(F_1 = 0.68)$, and *Fear of the Unknown* $(F_1 = 0.48)$
  - **Why?** Require cross-file context; imprecise localisation behaviour

RQ4: What patterns can be observed in the occurrence of SQL antipatterns in Java code that uses jOOQ for database access?

- 15,931 total antipatterns occurrences found in 602 projects, 26.5 per project
- **Dominant core:** "Implicit Columns" and "ID Required"
  - Contained by 89% and 87% of projects
- **Missing constraints:** "Keyless Entry" and "Beware of the Unknown"
  - Presence of one indicates the presence of other
- **Infection path:** Rarer SQL antipatterns are (weak) indicators of more common ones

RQ5: Which jOOQ API methods are most frequently associated with query antipattern occurrences?

Implicit Columns

- More than 75% associated with shorthands for fetching jOOQ records
  - e.g. `DSL.selectFrom(TABLE)`, `DSL.select().from(TABLE)`
- Only 12.9% explicitly show the intent to fetch all columns
  - e.g. `DSL.asterisk()`, `TABLE.fields()`

RQ5: Which jOOQ API methods are most frequently associated with query antipattern occurrences?

Poor Man's Search Engine

- More than 60% explicitly use `LIKE` and `ILIKE` operators with wildcards
  - `FIELD.like("%" + value + "%")`, `FIELD.ilike`
- Wildcards are added implicitly in 28% of cases
  - `FIELD.contains(value)`, `FIELD.containsIgnoreCase`

Highlights & Contributions

- Created an internally consistent ground truth
- First to treat LLM-based antipattern detection as a localisation task
- Developed the first available tool for detecting SQL antipatterns in jOOQ
- Performed a large-scale analysis of jOOQ projects

Thank you for listening!

1) Why have you not applied Abstract Syntax Trees at your SQL code classification method? How do you see the combination of LLMs and ASTs in SQL antipattern detection?

```
void fetchData() {
    return dslContext.select(TABLE.asterisk()).from(TABLE).fetchOne();
}
```

```
void fetchData(boolean fetchOther) {
    List<Field> columns = getColumns(fetchOther);
    return dslContext.select(columns).from(TABLE)
        .innerJoin(OTHER).on(OTHER.ID.eq(TABLE.OTHER_ID))
        .fetchOne();
}

void getColumns(boolean fetchOther) {
  List<Field> columns = new ArrayList<>();
  columns.add(TABLE.ID);
  if (fetchOther) columns.add(OTHER.asterisk());
  return columns;
}
```

```
public class UsersTable extends TableImpl<UsersTableRecord> {

  public final TableField<UsersTableRecord, String> GENDER =
    createField(DSL.name("gender"), SQLDataType.VARCHAR(255), this, "");

  @Override
  public List<Check<UsersTableRecord>> getChecks() {
    return Arrays.asList(
      Internal.createCheck(
        this,
        DSL.name("users_table_gender_check"),
        "(((gender)::text = ANY ((ARRAY['MALE'::character varying, 'FEMALE'::...])))",
        true
      )
    );
  }
}
```

2) Why some existing no-LLM detection solutions like DBDeo were not included in related work comparison?

- Comparison only included static analysis methods
  A. Performance of other methods was usually not evaluated
  B. Evaluation was under vastly different conditions
- First party evaluation of DbDeo did not provide any comparison points
- Third party study only provided precision: $P = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$

Antipattern	**This study**	**DbDeo**	**SQLCheck**
31 Flavors	0.97	0.52	0.99
Rounding Errors	1.00	0.99	0.98
…	…	…	…
Total	0.88	0.83	0.95

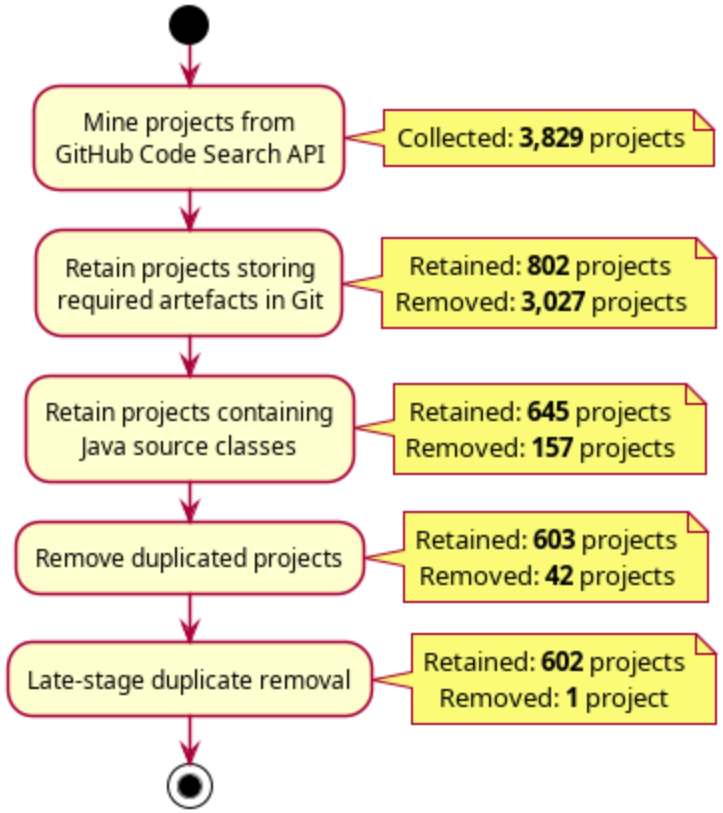
3) How would you overcome the size limitation of your original dataset? (Some antipattern types were not included in train-validate-test sets due to their rarity)

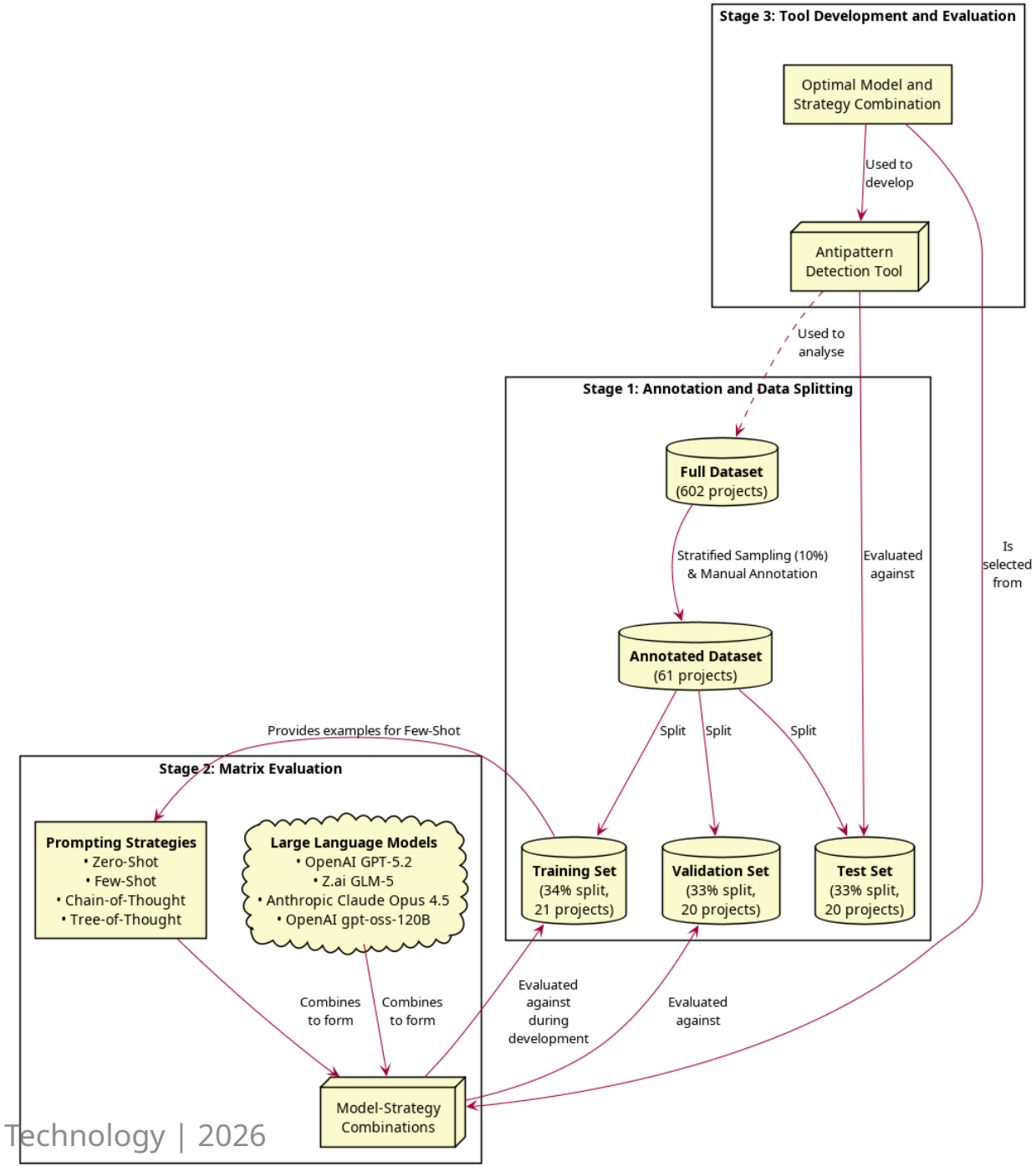
A. Complete shift of methodology

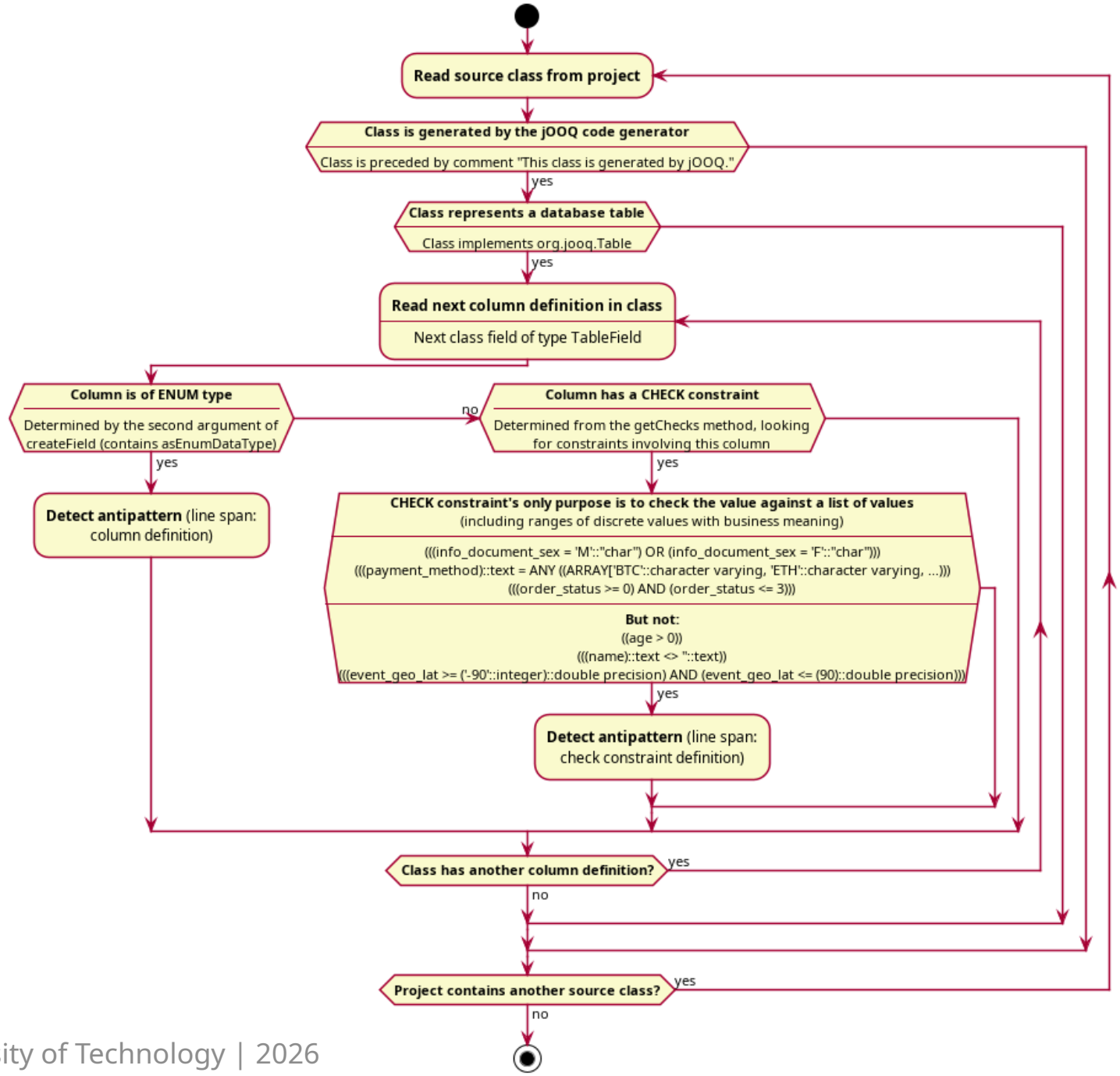
- Skip up-front annotation of ground truth
- Manually classify tool's detections as TP/FP
- Cache classifications, reuse for future iterations
- Replace recall ratio with absolute number of detections

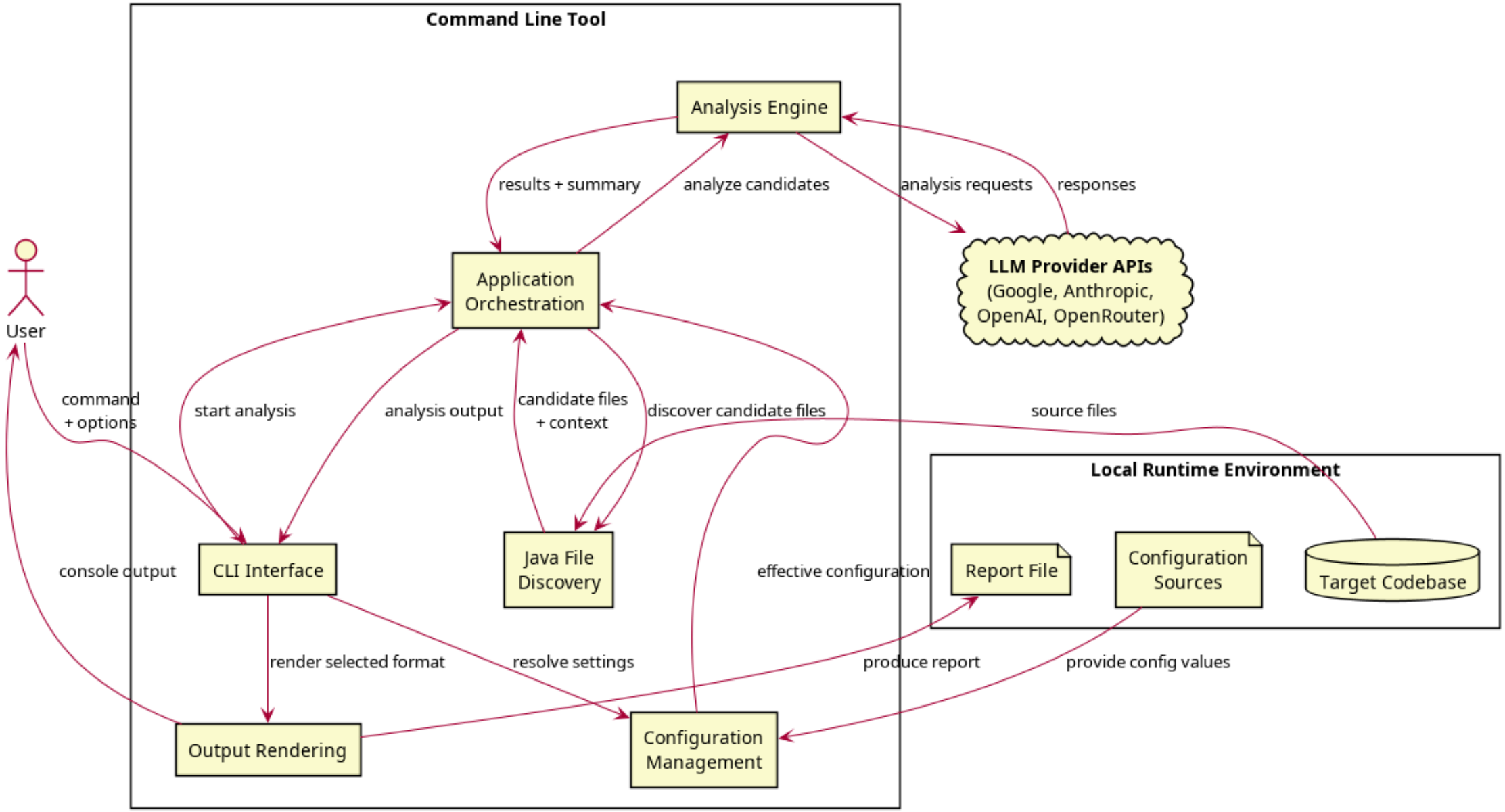
$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \quad R = \mathrm{TP}$$

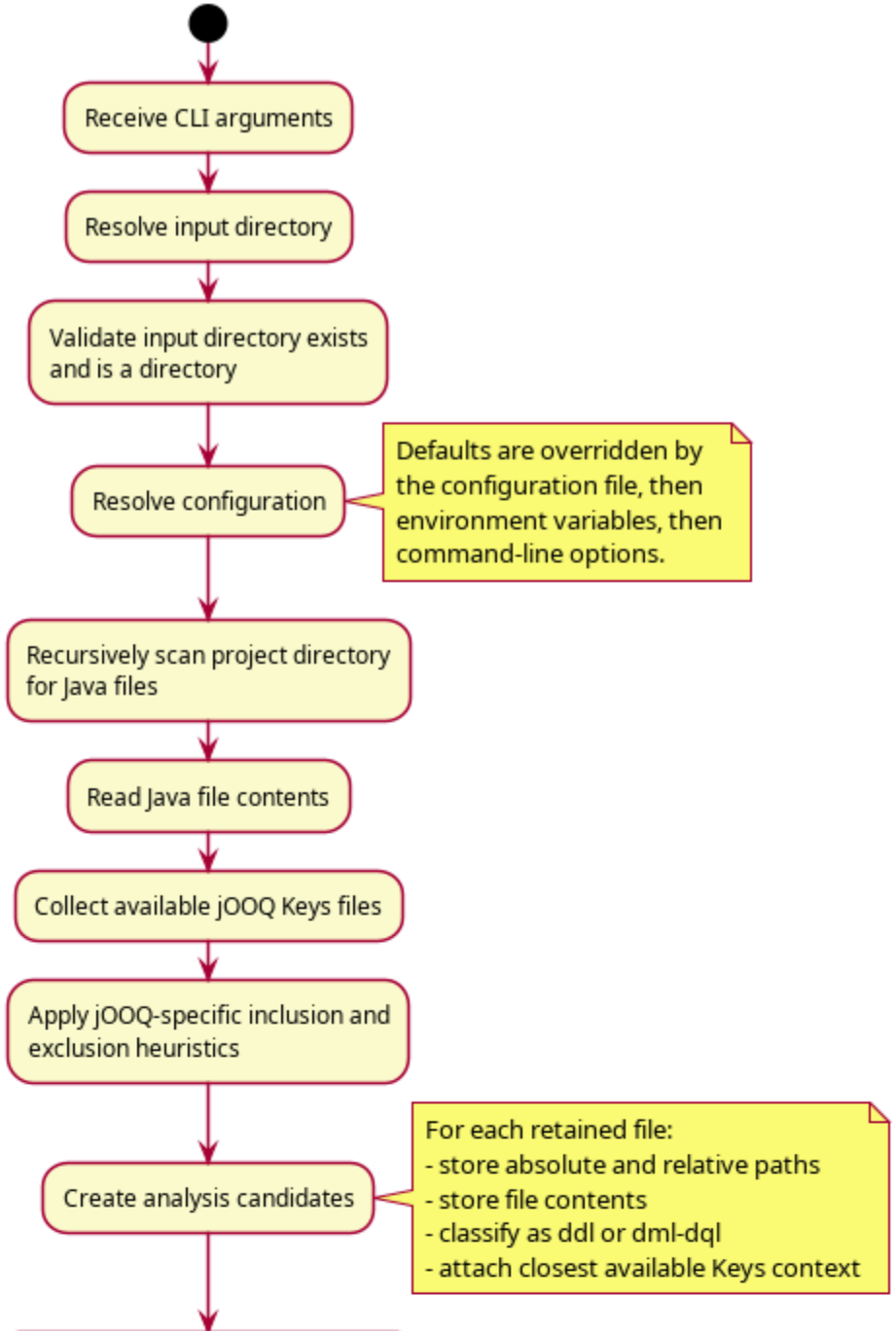
B. Augment data with synthetic samples

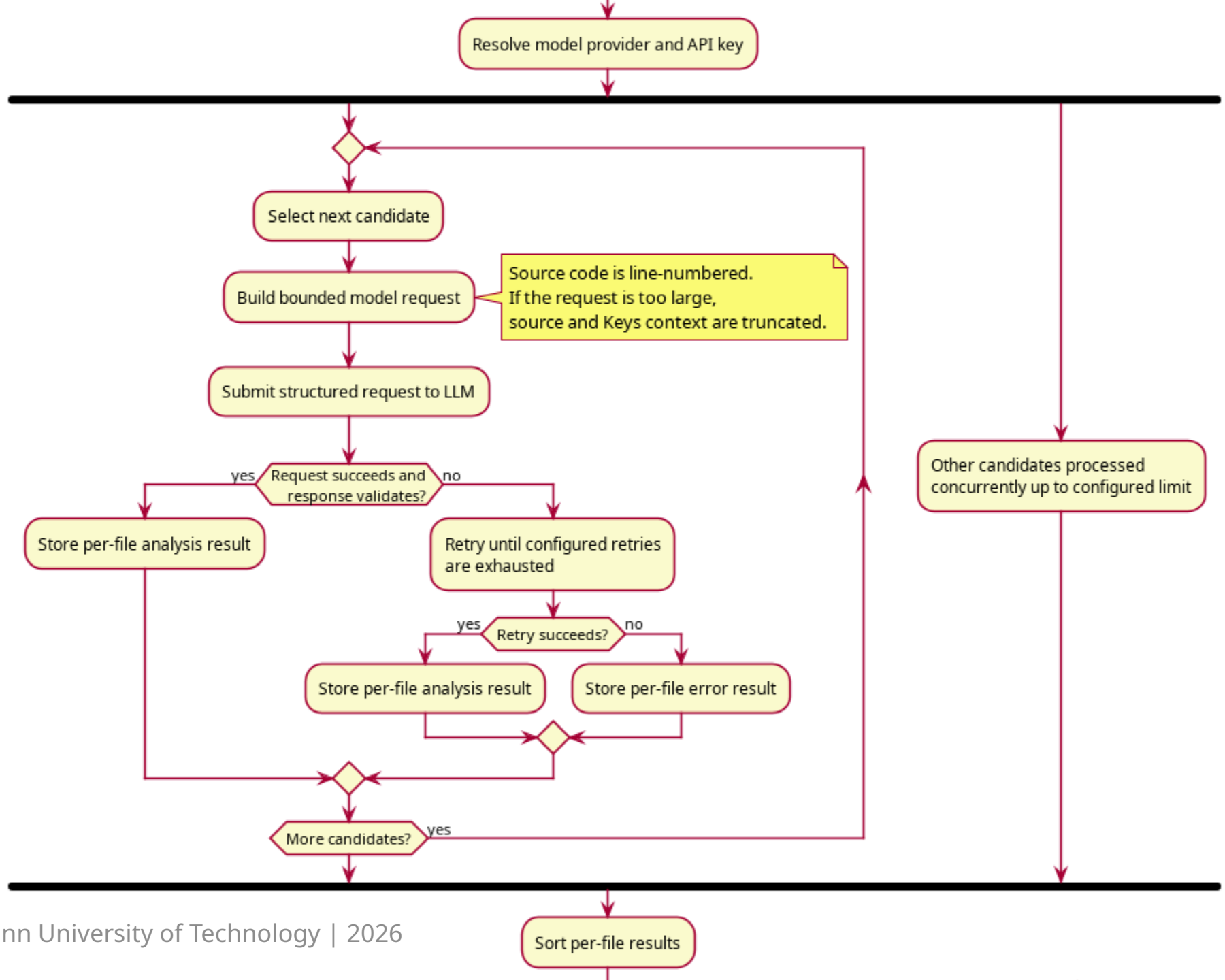

Mine projects from GitHub Code Search API
Collected: 3,829 projects
Retain projects storing required artefacts in Git
Retained: 802 projects
Removed: 3,027 projects
Retain projects containing Java source classes
Retained: 645 projects
Removed: 157 projects
Remove duplicated projects
Retained: 603 projects
Removed: 42 projects
Late-stage duplicate removal
Retained: 602 projects
Removed: 1 project

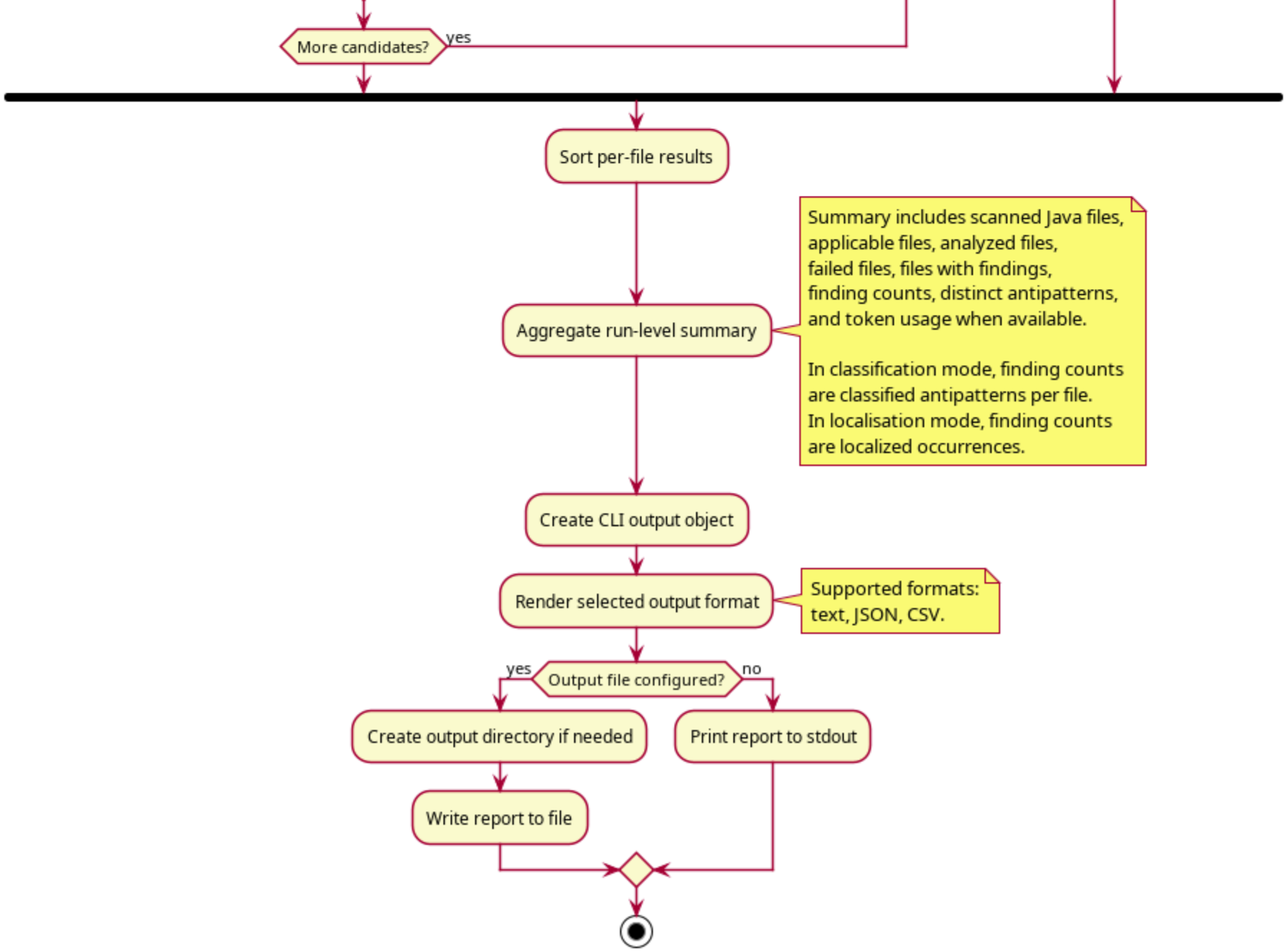

Stage 3: Tool Development and Evaluation
Optimal Model and Strategy Combination
Used to develop
Antipattern Detection Tool
Used to analyse
Is selected from
Stage 1: Annotation and Data Splitting
Full Dataset (602 projects)
Stratified Sampling (10%) & Manual Annotation
Evaluated against
Annotated Dataset (61 projects)
Split
Split
Split
Training Set (34% split, 21 projects)
Validation Set (33% split, 20 projects)
Test Set (33% split, 20 projects)
Provides examples for Few-Shot
Stage 2: Matrix Evaluation
Prompting Strategies
• Zero-Shot
• Few-Shot
• Chain-of-Thought
• Tree-of-Thought
Large Language Models
• OpenAI GPT-5.2
• Z.ai GLM-5
• Anthropic Claude Opus 4.5
• OpenAI gpt-oss-120B
Combines to form
Combines to form
Evaluated against during development
Evaluated against
Model-Strategy Combinations


Read source class from project
Class is generated by the jOOQ code generator
Class is preceded by comment "This class is generated by jOOQ."
yes
Class represents a database table
Class implements org.jooq.Table
yes
Read next column definition in class
Next class field of type TableField
Column is of ENUM type
Determined by the second argument of createField (contains asEnumDataType)
yes
Detect antipattern (line span: column definition)
no
Column has a CHECK constraint
Determined from the getChecks method, looking for constraints involving this column
yes
CHECK constraint's only purpose is to check the value against a list of values
(including ranges of discrete values with business meaning)
(((info_document_sex = 'M'::"char") OR (info_document_sex = 'F'::"char")))
(((payment_method)::text = ANY ((ARRAY['BTC'::character varying, 'ETH'::character varying, ...)))
(((order_status >= 0) AND (order_status <= 3)))
But not:
((age > 0))
(((name)::text <> ''::text))
(((event_geo_lat >= ('-90'::integer)::double precision) AND (event_geo_lat <= (90)::double precision)))
yes
Detect antipattern (line span: check constraint definition)
Class has another column definition?
yes
no
Project contains another source class?
yes
no


Command Line Tool
Analysis Engine
results + summary
analyze candidates
analysis requests
responses
Application Orchestration
LLM Provider APIs
(Google, Anthropic, OpenAI, OpenRouter)
User
command + options
start analysis
analysis output
candidate files + context
discover candidate files
source files
Local Runtime Environment
console output
CLI Interface
Java File Discovery
effective configuration
Report File
Configuration Sources
Target Codebase
render selected format
resolve settings
produce report
provide config values
Output Rendering
Configuration Management


Receive CLI arguments
Resolve input directory
Validate input directory exists and is a directory
Resolve configuration
Defaults are overridden by the configuration file, then environment variables, then command-line options.
Recursively scan project directory for Java files
Read Java file contents
Collect available jOOQ Keys files
Apply jOOQ-specific inclusion and exclusion heuristics
Create analysis candidates
For each retained file:
- store absolute and relative paths
- store file contents
- classify as ddl or dml-dql
- attach closest available Keys context


Resolve model provider and API key
Select next candidate
Build bounded model request
Source code is line-numbered.
If the request is too large,
source and Keys context are truncated.
Submit structured request to LLM
yes
Request succeeds and response validates?
no
Store per-file analysis result
Retry until configured retries are exhausted
yes
Retry succeeds?
no
Store per-file analysis result
Store per-file error result
More candidates?
yes
Other candidates processed concurrently up to configured limit
Sort per-file results


More candidates?
yes
Sort per-file results
Aggregate run-level summary
Summary includes scanned Java files, applicable files, analyzed files, failed files, files with findings, finding counts, distinct antipatterns, and token usage when available.
In classification mode, finding counts are classified antipatterns per file.
In localisation mode, finding counts are localized occurrences.
Create CLI output object
Render selected output format
Supported formats: text, JSON, CSV.
yes
Output file configured?
no
Create output directory if needed
Print report to stdout
Write report to file

RQ3: How accurately can the developed LLM-based tool detect SQL antipatterns in Java code that uses jOOQ for database access?

Antipattern	Precision	Recall	F1-Score
Implicit Columns	0.98	0.94	0.96
ID Required	0.87	0.85	0.86
Keyless Entry	0.54	0.97	0.69
Fear of the Unknown	0.44	0.53	0.48
31 Flavors	0.97	0.97	0.97
Poor Man's Search Engine	0.76	0.62	0.68
Rounding Errors	1.00	0.81	0.90
Total	**0.88**	**0.88**	**0.88**